

2025년 상반기 K-디지털 트레이닝

JDBC 프로그래밍

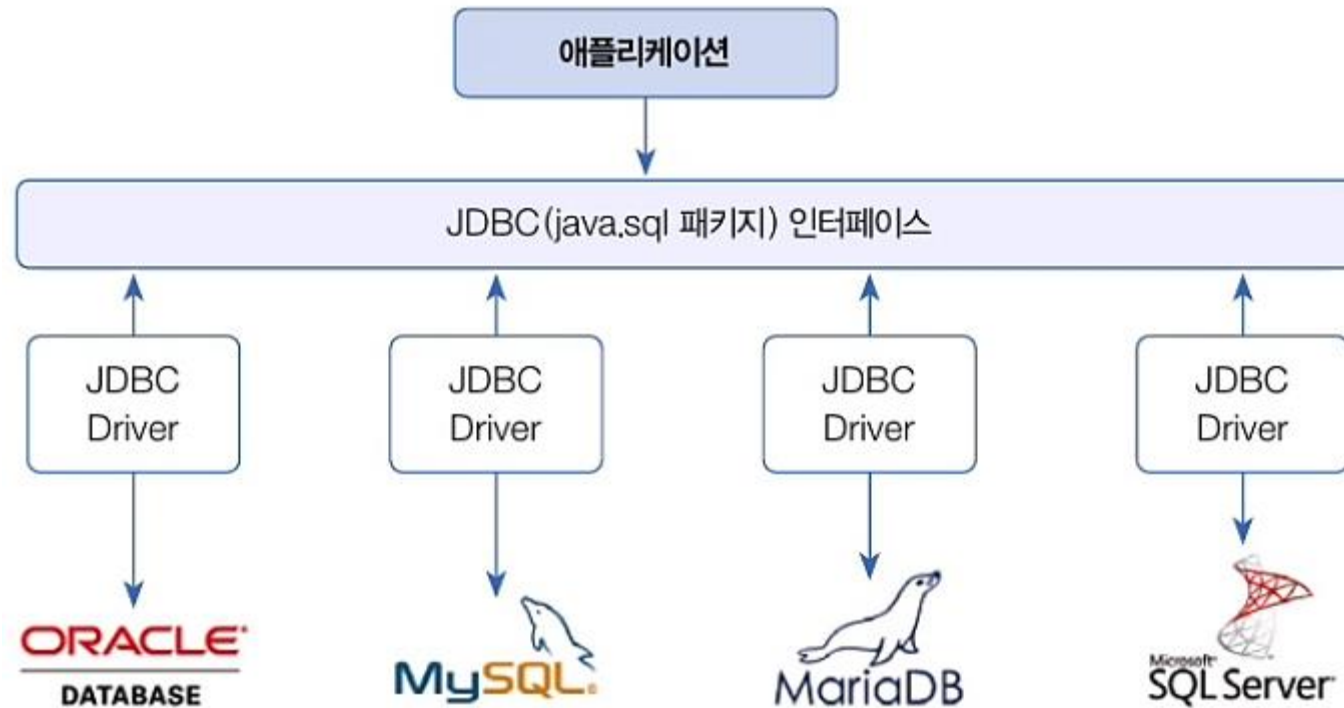
[KB] IT's Your Life



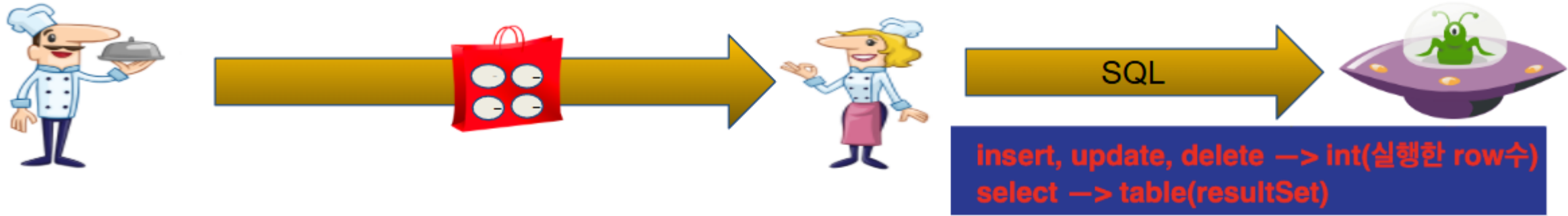
1 JDBC 프로그래밍

✓ JDBC Java Database Connectivity

- 데이터베이스와 연결해서 입출력을 지원
- 데이터베이스 관리시스템(DBMS)의 종류와 상관없이 동일하게 사용할 수 있는 클래스와 인터페이스로 구성



1 Application 구조(응집도)



자바프로그램(역할- UI)

나의 네이버 회원가입

NAVER

ID

PW

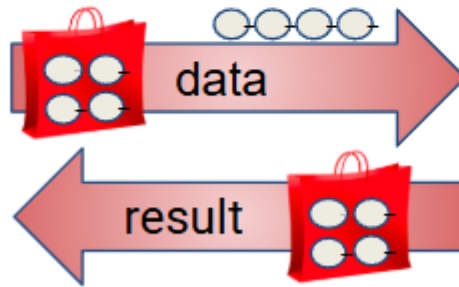
NAME

TEL

회원가입완료

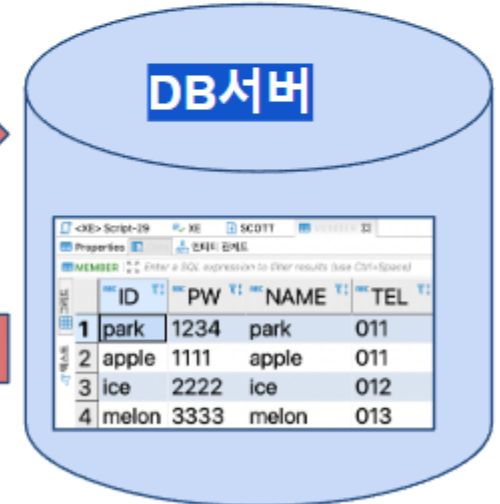
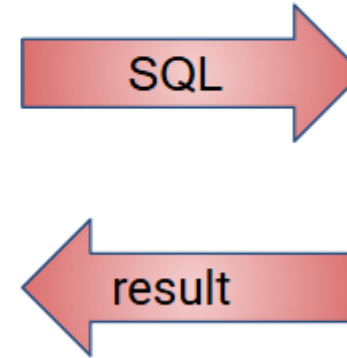
회원탈퇴처리

자바프로그램 (역할- 가방, VO)



자바프로그램 (역할- DB관련 처리, DAO)

- 1) 사용할 드라이버 설정
- 2) DB연결
- 3) SQL문 생성
- 4) mysql 서버로 SQL전송



1 JDBC 프로그래밍

```

INSERT INTO "MEMBER"
VALUES ('com', 'com', 'com', 'com')

UPDATE "MEMBER" SET tel = '999'
WHERE id = 'com'

DELETE FROM "MEMBER" m
WHERE id = 'com'

```

Statistics 1

Name	Value
Updated Rows	1
Query	DELETE FROM "MEMBER" m WHERE id = 'com'
Finish time	Mon Mar 06 16:21:13 KST 2023

CUD의 결과는
실행된 row의
개수(정수)

R의 결과는 실행된
테이블 형태
(컬럼+인스턴스)

```
SELECT * FROM "MEMBER" m
```

Results 1

SELECT * FROM "MEMBER" m

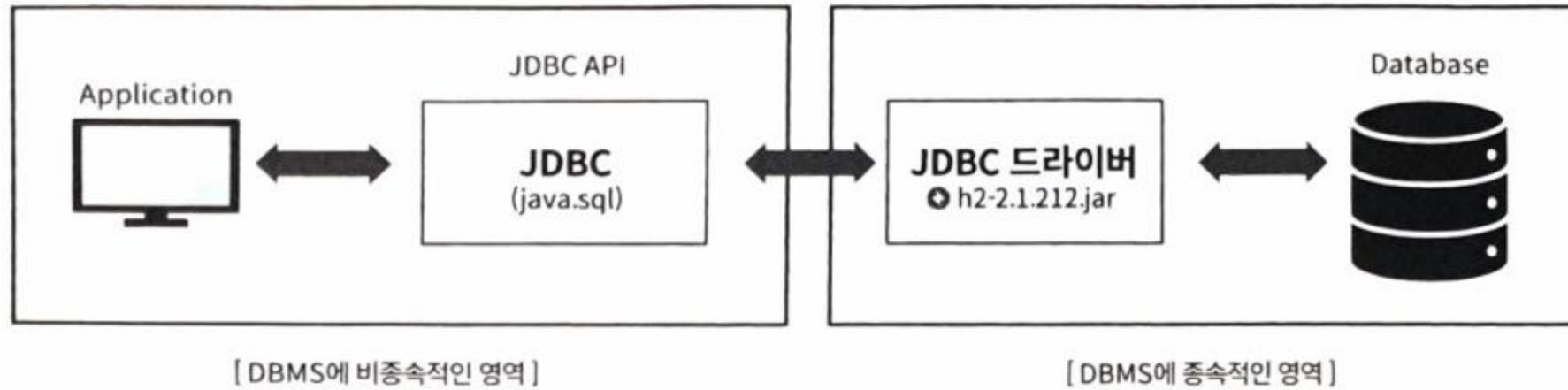
	ID	PW	NAME	TEL
1	apple	apple	apple	apple
2	ice	ice	ice	ice
3	kiwi	kiwi	kiwi	[NULL]

DB처리 담당 클래스	데이터 전달 담당 클래스
-Data Access Object(DAO) DB 접근해서 처리하는 클래스 데이터를 가지고 할 수 있는 일 Create, Read, Update, Delete(CRUD)	-Data Transfer Object(DTO) ,VO 입력한 데이터를 DB전송하기 위해서 데이터를 묶기 위한 가방역할의 클래스

1 JDBC 프로그래밍

✓ JDBC

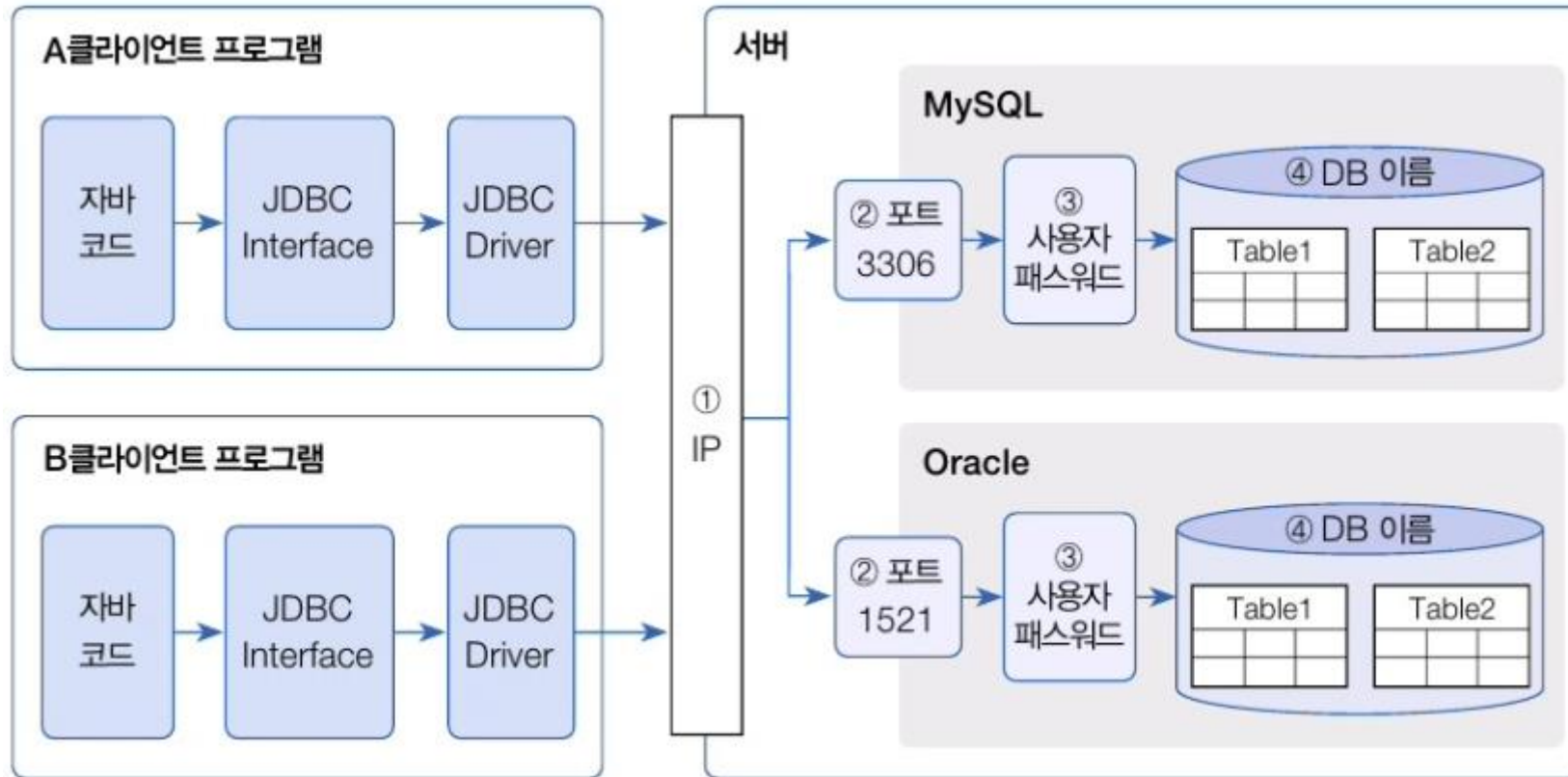
○ JDBC 개념



1 JDBC 프로그래밍

✓ JDBC

○ JDBC 개념



1 JDBC 프로그래밍

✓ 데이터베이스 준비

```
CREATE DATABASE jdbc_ex;
```

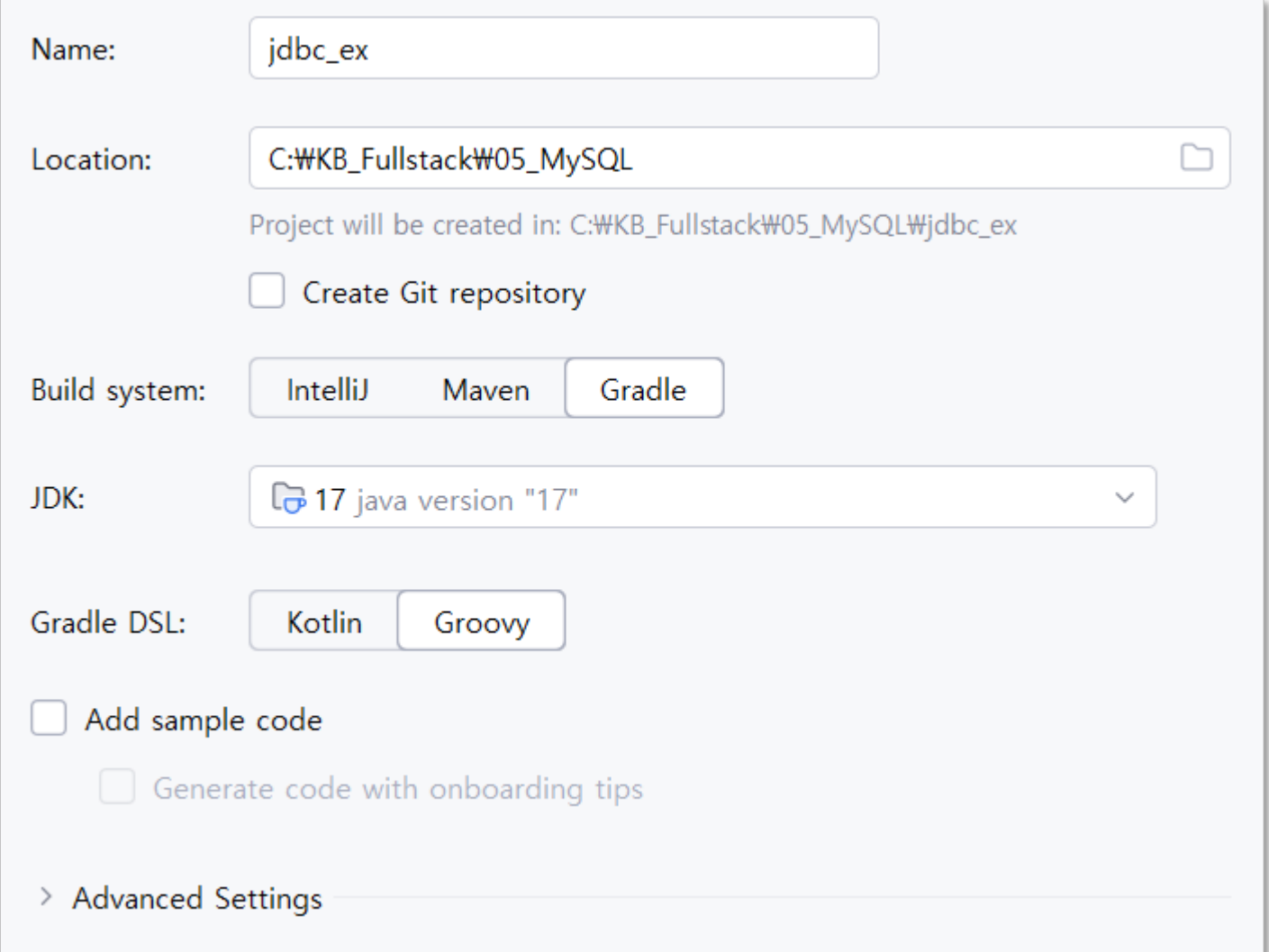
✓ 사용자 준비

```
CREATE USER 'scoula'@'%' IDENTIFIED BY '1234';  
GRANT ALL PRIVILEGES ON jdbc_ex.* TO 'scoula'@'%';  
FLUSH PRIVILEGES;
```


1 JDBC 프로그래밍

✓ 프로젝트 생성

- Name: jdbc_ex
- Build System : gradle
- Group Id: org.scoula



The image shows the 'New Project' dialog box in IntelliJ IDEA. The 'Name' field is set to 'jdbc_ex'. The 'Location' field is set to 'C:\WKB_Fullstack\W05_MySQL', with a subtext indicating the project will be created in 'C:\WKB_Fullstack\W05_MySQL\jdbc_ex'. The 'Build system' is set to 'Gradle' (with 'IntelliJ' and 'Maven' as alternatives). The 'JDK' is set to '17 java version "17"'. The 'Gradle DSL' is set to 'Kotlin' (with 'Groovy' as an alternative). There are checkboxes for 'Create Git repository' (unchecked), 'Add sample code' (unchecked), and 'Generate code with onboarding tips' (unchecked). An 'Advanced Settings' link is at the bottom.

Name: jdbc_ex

Location: C:\WKB_Fullstack\W05_MySQL
Project will be created in: C:\WKB_Fullstack\W05_MySQL\jdbc_ex

☐ Create Git repository

Build system: IntelliJ Maven **Gradle**

JDK: 17 java version "17"

Gradle DSL: Kotlin **Groovy**

☐ Add sample code
☐ Generate code with onboarding tips

> Advanced Settings

1 JDBC 프로그래밍

✓ MySQL Connector

- <https://mvnrepository.com/>
- MySQL 검색



1. MySQL Connector/J 585 usages

[com.mysql » mysql-connector-j](#)

MySQL Connector/J is a JDBC Type 4 driver, which means that it is pure Java implementation of the MySQL protocol and does not rely on the registration with the Driver Manager, standard support for large update counts, support for local java.time package, support for JDBC-4.x XML protocol information and support for the NCHAR, NVARCHAR

Last Release on Jan 16, 2024

Version	Vulnerabilities	Repository	Usages	Date
8.3.x 8.3.0		Central	168	Jan 16, 2024
8.2.x 8.2.0		Central	127	Oct 25, 2023
8.1.x 8.1.0		Central	164	Jul 18, 2023
8.0.33		Central	279	Apr 18, 2023
8.0.x 8.0.32		Central	139	Jan 18, 2023
		Central	106	Oct 14, 2022

[Maven](#)
[Gradle](#)
[Gradle \(Short\)](#)
[Gradle \(Kotlin\)](#)
[SBT](#)
[Ivy](#)
[Grape](#)
[Leiningen](#)
[Buildr](#)

```
// https://mvnrepository.com/artifact/com.mysql/mysql-connector-j
implementation 'com.mysql:mysql-connector-j:8.3.0'
```

 build.gradle

```
...
dependencies {
    implementation 'com.mysql:mysql-connector-j:8.3.0'
    compileOnly 'org.projectlombok:lombok:1.18.30'
    annotationProcessor 'org.projectlombok:lombok:1.18.30'

    testCompileOnly 'org.projectlombok:lombok:1.18.30'
    testAnnotationProcessor 'org.projectlombok:lombok:1.18.30'

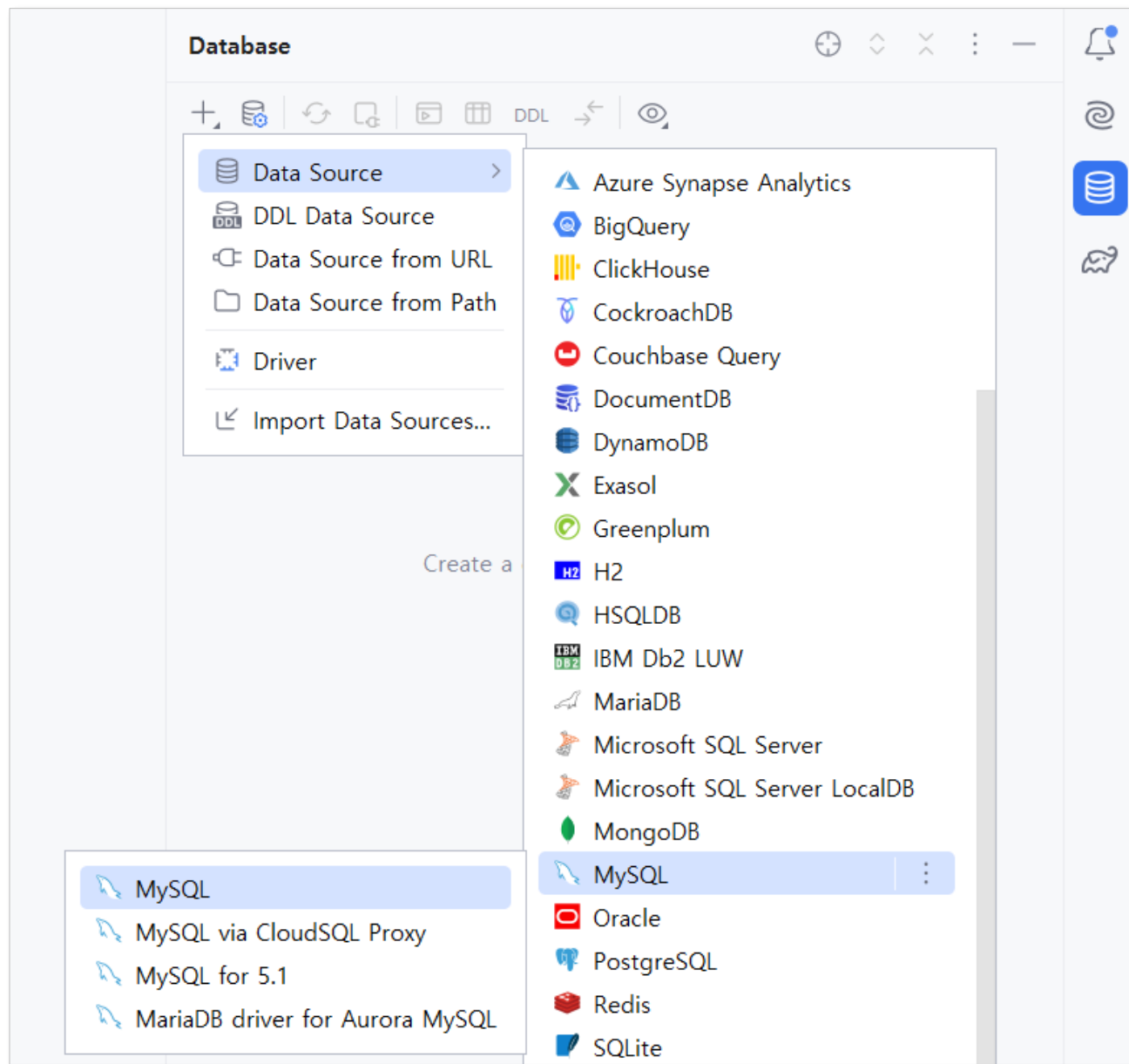
    testImplementation platform('org.junit:junit-bom:5.9.1')
    testImplementation 'org.junit.jupiter:junit-jupiter'
}
...
```

○ 수정 후 Sync 실행

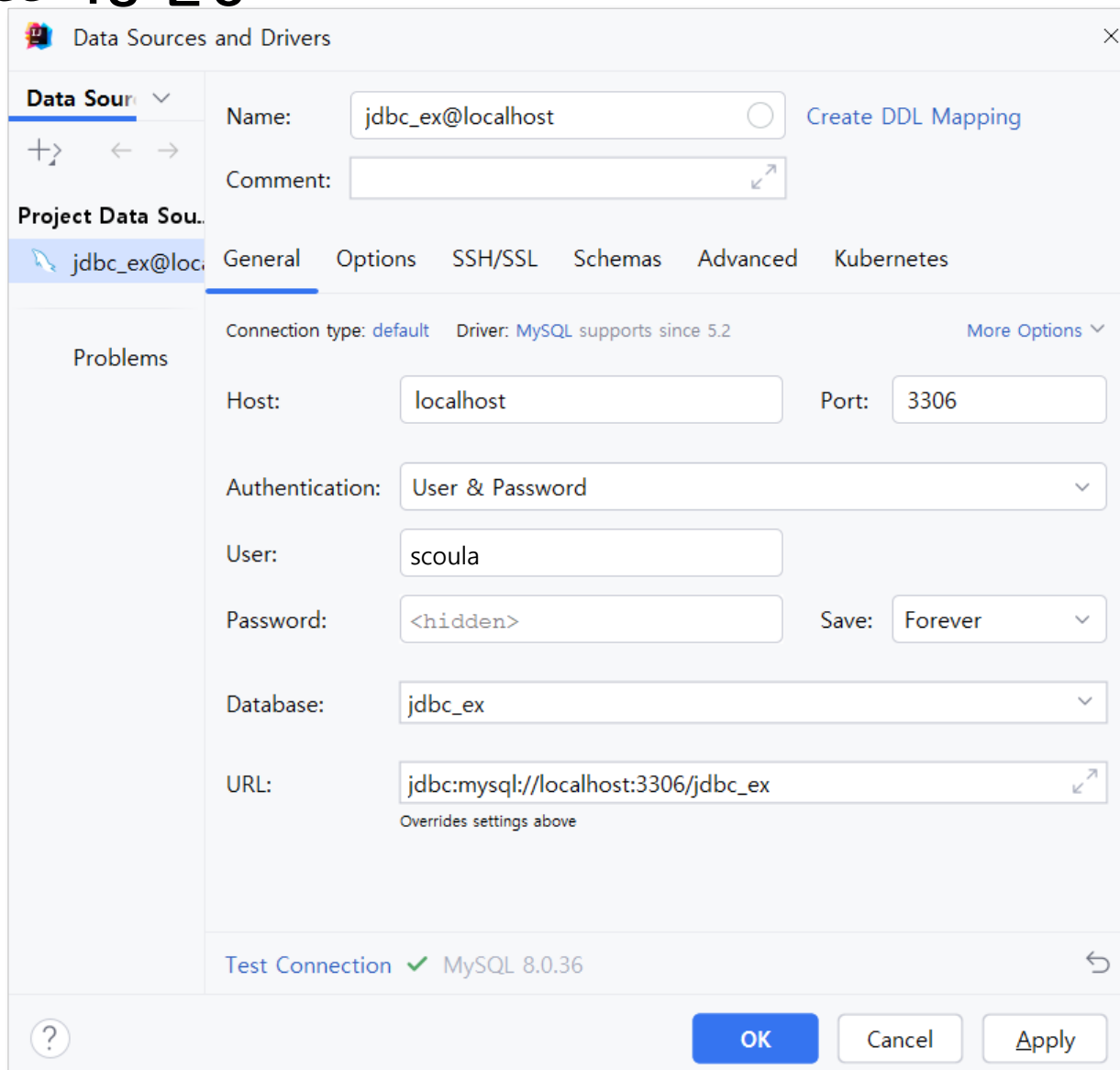
○ 프로젝트 설정

- Annotation Processing 활성화

✓ IntelliJ Datasource 기능 설정



✓ IntelliJ Datasource 기능 설정

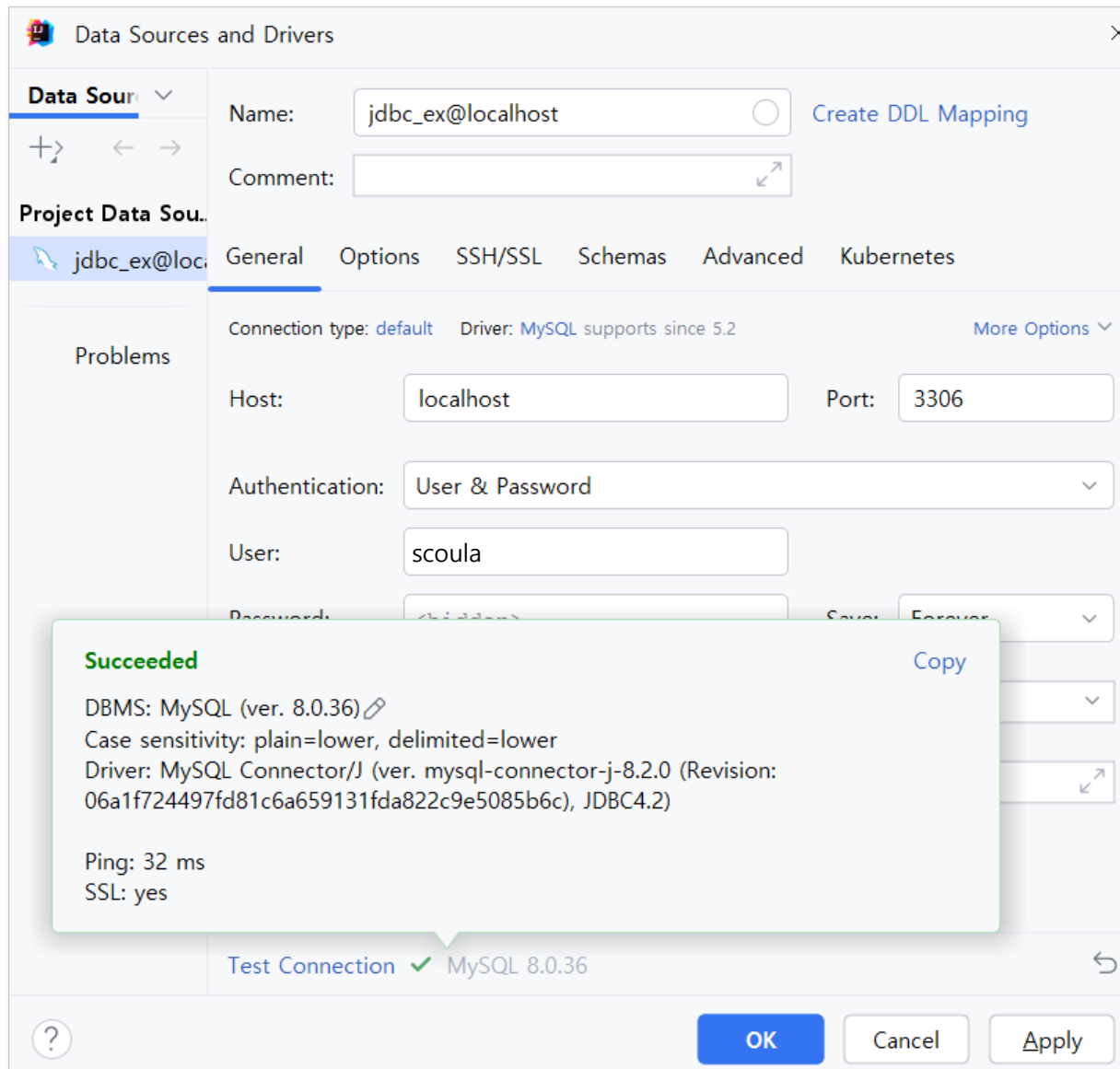


The image shows the 'Data Sources and Drivers' dialog box in IntelliJ IDEA. The 'Data Sources' tab is selected, and a new data source named 'jdbc_ex@localhost' is being configured. The 'General' tab is active, showing the following settings:

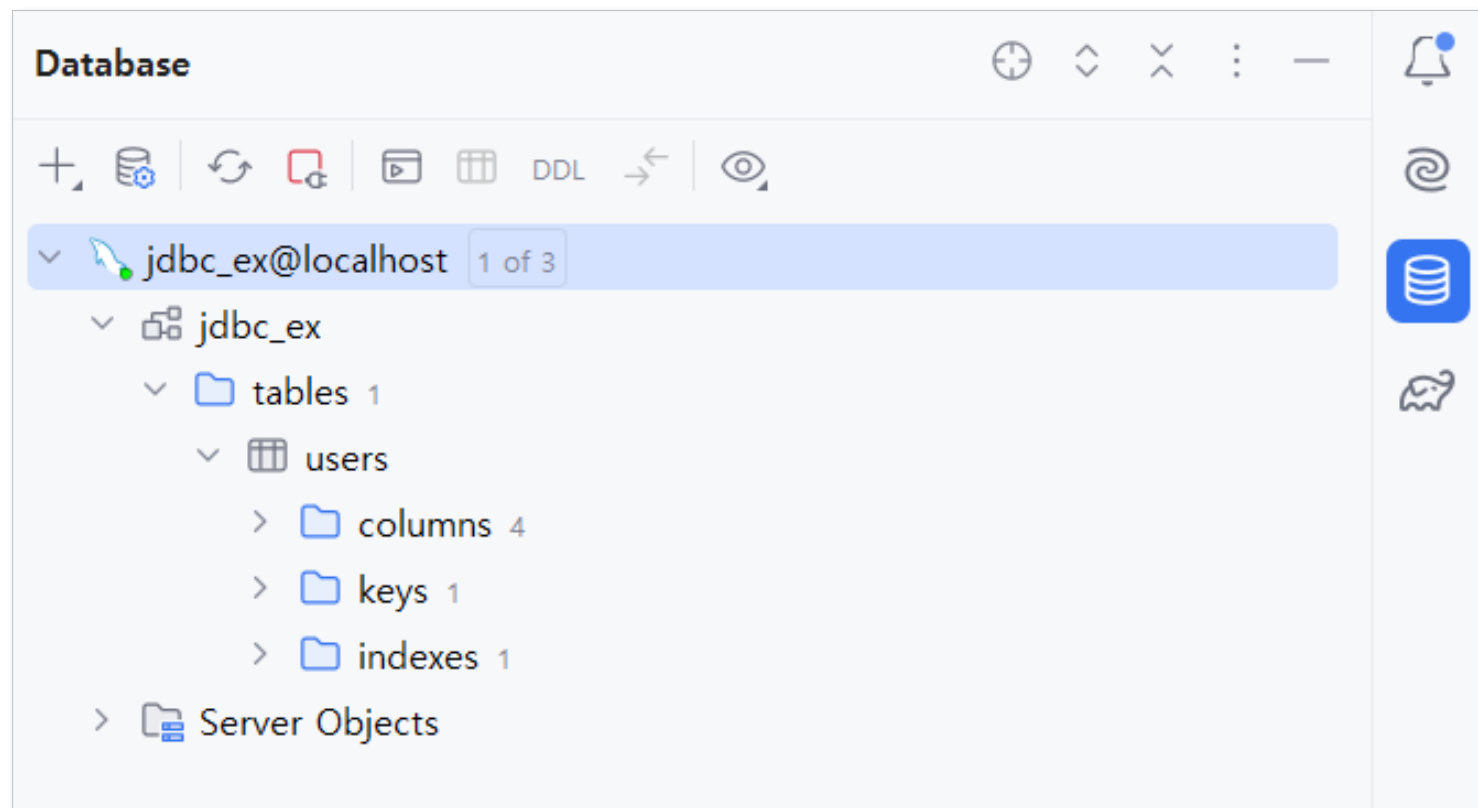
- Name: jdbc_ex@localhost
- Comment: (empty)
- Connection type: default
- Driver: MySQL supports since 5.2
- Host: localhost
- Port: 3306
- Authentication: User & Password
- User: scoula
- Password: <hidden>
- Save: Forever
- Database: jdbc_ex
- URL: jdbc:mysql://localhost:3306/jdbc_ex

The 'Test Connection' button is highlighted, and the status shows 'MySQL 8.0.36'.

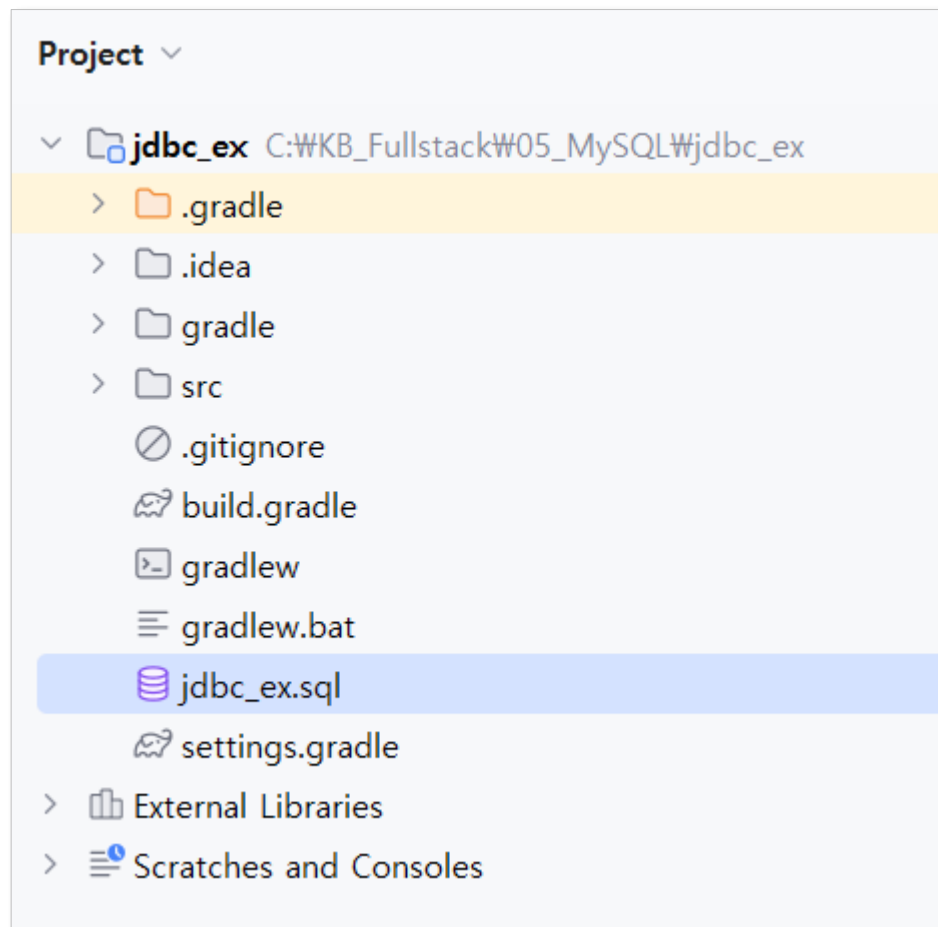
✓ IntelliJ Datasource 기능 설정



✓ datasource 목록



✓ sql 파일 만들기



✓ 쿼리 실행

1. datasource 선택

2. schema(database) 선택

4. 쿼리 실행

3. 쿼리 작성

```
select * from users;
```

Output: jdbc_ex.users

ID	PASSWORD	NAME	ROLE	
1	admin	admin123	관리자	ADMIN
2	guest	guest123	방문자	USER
3	member	member123	일반회원	USER

1 JDBC 프로그래밍

✓ 데이터 준비

```
use jdbc_ex;
```

```
CREATE TABLE USERS (  
  ID VARCHAR(12) NOT NULL PRIMARY KEY,  
  PASSWORD VARCHAR(12) NOT NULL,  
  NAME VARCHAR(30) NOT NULL,  
  ROLE VARCHAR(6) NOT NULL  
);
```

```
INSERT INTO USERS(ID, PASSWORD, NAME, ROLE)  
VALUES('guest', 'guest123', '방문자', 'USER');
```

```
INSERT INTO USERS(ID, PASSWORD, NAME, ROLE)  
VALUES('admin', 'admin123', '관리자', 'ADMIN');
```

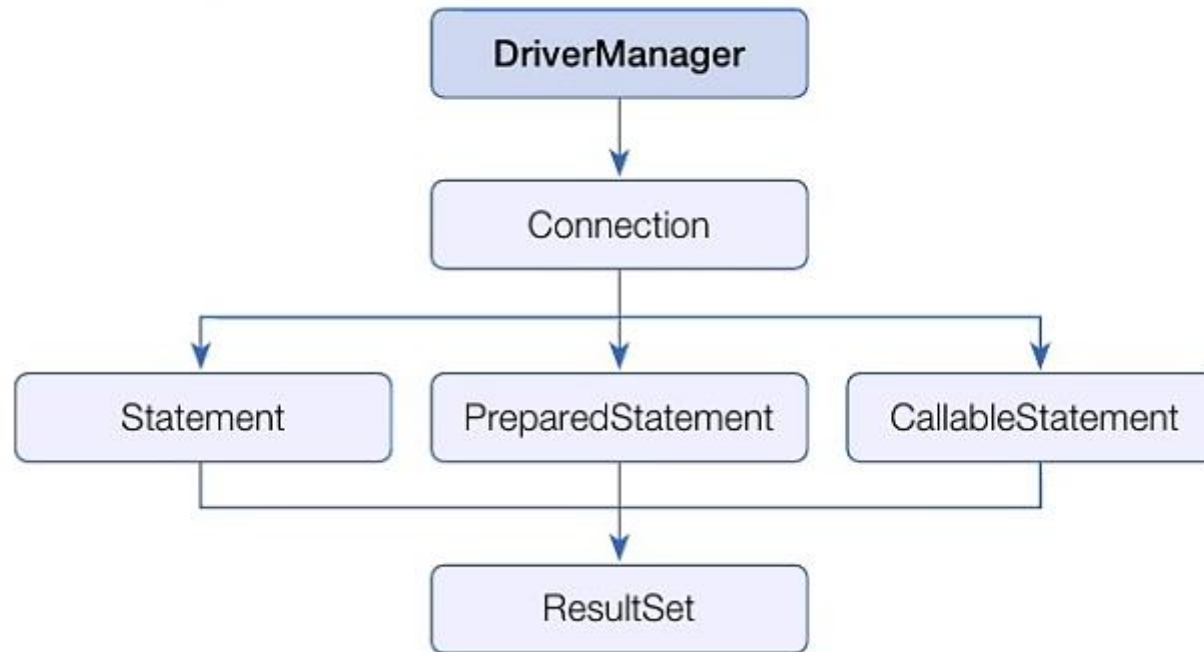
```
INSERT INTO USERS(ID, PASSWORD, NAME, ROLE)  
VALUES('member', 'member123', '일반회원', 'USER');
```

```
SELECT * FROM USERS;
```

1 JDBC 프로그래밍

✓ JDBC의 핵심 인터페이스/클래스

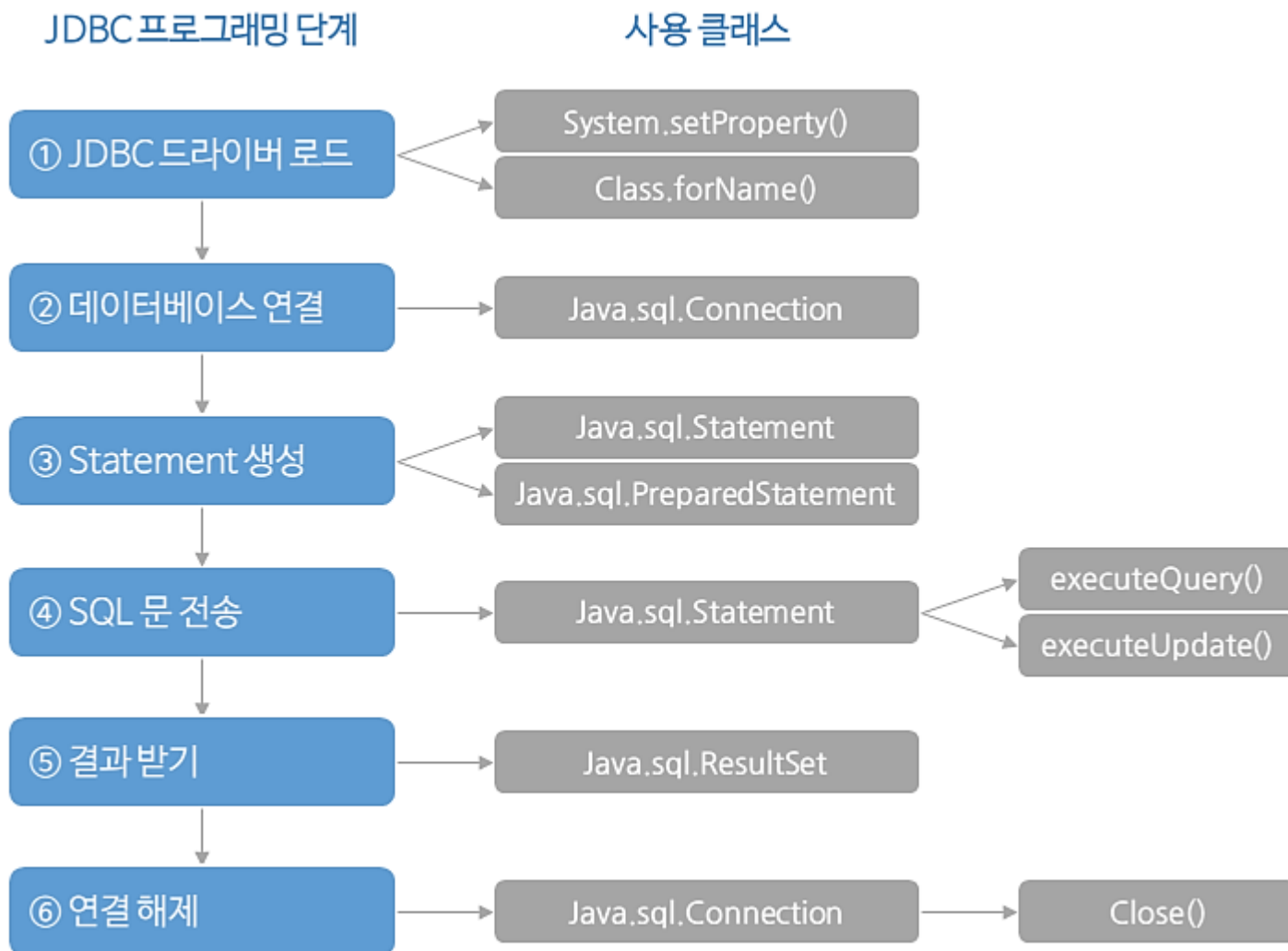
JDBC (java.sql 패키지)



1 JDBC 프로그래밍

✓ JDBC

○ JDBC 개발 절차

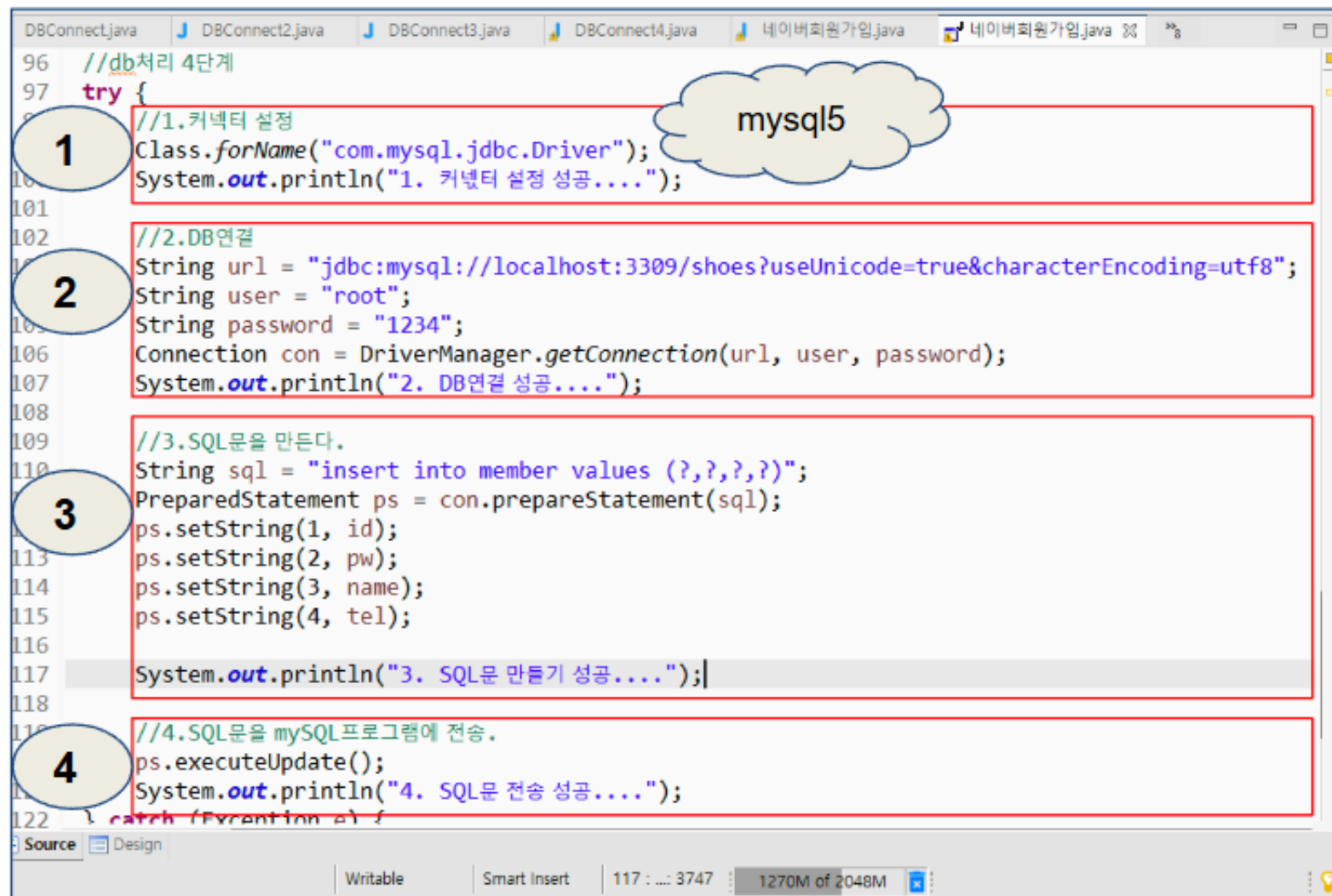


1 JDBC 프로그래밍

DB 드라이버 설정

DB 연결

SQL문 객체 생성

DB로 보내 실행
/결과처리

```
DBConnect.java DBConnect2.java DBConnect3.java DBConnect4.java 네이버회원가입.java 네이버회원가입.java
96 //db처리 4단계
97 try {
98     //1.커넥터 설정
99     Class.forName("com.mysql.jdbc.Driver");
100     System.out.println("1. 커넥터 설정 성공....");
101
102     //2.DB연결
103     String url = "jdbc:mysql://localhost:3309/shoes?useUnicode=true&characterEncoding=utf8";
104     String user = "root";
105     String password = "1234";
106     Connection con = DriverManager.getConnection(url, user, password);
107     System.out.println("2. DB연결 성공....");
108
109     //3.SQL문을 만든다.
110     String sql = "insert into member values (?, ?, ?, ?)";
111     PreparedStatement ps = con.prepareStatement(sql);
112     ps.setString(1, id);
113     ps.setString(2, pw);
114     ps.setString(3, name);
115     ps.setString(4, tel);
116
117     System.out.println("3. SQL문 만들기 성공....");
118
119     //4.SQL문을 mySQL 프로그램에 전송.
120     ps.executeUpdate();
121     System.out.println("4. SQL문 전송 성공....");
122 } catch (Exception e) {
    e.printStackTrace();
}
```

mysql5

1 JDBC 프로그래밍

✓ 테스트

```
public class DBConnect_mySQL {
    public static void main(String[] args) {
        Connection con = null;
        PreparedStatement ps = null;
        try {
            // 1. 드라이버 설정
            Class.forName("com.mysql.cj.jdbc.Driver");
            System.out.println("1. 드라이버 설정 성공..");

            // 2. 데이터베이스 연결
            String url = "jdbc:mysql://localhost:3306/jdbc_ex";
            String user = "scoula";
            String password = "1234";
            con = DriverManager.getConnection(url, user, password);
            System.out.println("2. db연결 성공.");

            // 3. SQL 문 객체 생성
            String sql = "insert into users
                values ('win', win', 'win', 'USER')";
            ps = con.prepareStatement(sql);
            System.out.println("3. sql문 객체 생성 성공.");
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            if (ps != null) ps.close();
            if (con != null) con.close();
        }
    }
}
```

// 4. SQL 문 실행 및 결과 출력

```
int result = ps.executeUpdate();
System.out.println("4. sql문 전송 성공, 결과>> " + result);
} catch (ClassNotFoundException e) {
    System.out.println("드라이버 로드 실패: " + e.getMessage());
} catch (SQLException e) {
    System.out.println("DB실패: " + e.getMessage());
}finally {
    // 자원 해제
    try {
        if (ps != null) ps.close();
        if (con != null) con.close();
    } catch (SQLException e) {
        System.out.println("자원 해제 실패: " + e.getMessage());
    }
}
}
```

1 JDBC 프로그래밍

✓ DB 연결

○ 드라이버 확인

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

→ 없으면 ClassNotFoundException 발생

○ Connection 객체

- 데이터베이스에 연결 세션을 만듦
- `Connection conn = DriverManager.getConnection("연결 문자열", "사용자", "비밀번호")`
- 연결 문자열
`"jdbc:mysql://[host]:[포트]/[db이름]"`
→ `jdbc:mysql://127.0.0.1:3306/jdbc_ex`
- `String url = "jdbc:mysql://127.0.0.1:3306/jdbc_ex";`
- `Connection conn = DriverManager.getConnection(url, "jdbc_ex", "jdbc_ex");`

ConnectionTest.java

```
package org.scoula.jdbc_ex.test;

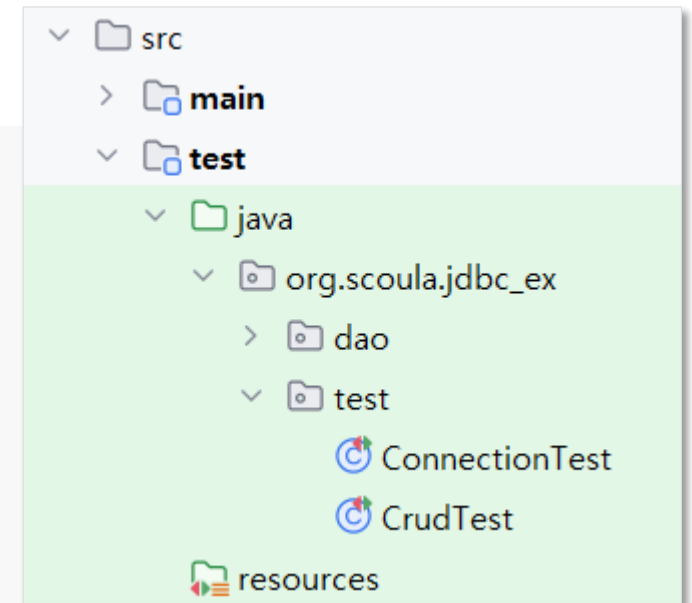
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConnectionTest {

    @Test
    @DisplayName("jdbc_ex 데이터베이스에 접속한다.")
    public void testConnection() throws SQLException, ClassNotFoundException {
        Class.forName("com.mysql.cj.jdbc.Driver");
        String url = "jdbc:mysql://127.0.0.1:3306/jdbc_ex";
        String id = "scoula";
        String password = "1234";
        Connection conn = DriverManager.getConnection(url, id, password);
        System.out.println("DB 연결 성공");
        conn.close();
    }
}
```

DB 연결 성공



1 JDBC 프로그래밍

✓ 모듈화해야 할 코드

- 데이터베이스 연결 및 닫기 작업은 항상 필요함

→ `common.JDBCUtil`

 resources:/application.properties

```
driver=com.mysql.cj.jdbc.Driver  
url=jdbc:mysql://127.0.0.1:3306/jdbc_ex  
id=scoula  
password=1234
```

 JDBCUtil.java

```
package org.scoula.jdbc_ex.common;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.util.Properties;

public class JDBCUtil {
    static Connection conn = null;
    public static Connection getConnection() {
        if(conn != null) return conn;
        try {
            Properties properties = new Properties();
            properties.load(JDBCUtil.class.getResourceAsStream("/application.properties"));
            String driver = properties.getProperty("driver");
            String url = properties.getProperty("url");
            String id = properties.getProperty("id");
            String password = properties.getProperty("password");

            Class.forName(driver);
            conn = DriverManager.getConnection(url, id, password);
        } catch (Exception e) {
            e.printStackTrace();
        }

        return conn;
    }
}
```

1 JDBC 프로그래밍

 JDBCUtil.java

```
public static void close() {  
    try {  
        if (conn != null) {  
            conn.close();  
            conn = null;  
        }  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

네트워크 연결
을 닫으므로,
try-catch필요

1 JDBC 프로그래밍

 ConnectionTest.java

```
package org.scoula.jdbc_ex.test;
...

public class ConnectionTest {

    ...
    @Test
    @DisplayName("jdbc_ex에 접속한다. (자동 닫기)")
    public void testConnection2() throws SQLException {
        try(Connection conn = JDBCUtil.getConnection()) {
            System.out.println("DB 연결 성공");
        }
    }
}
```

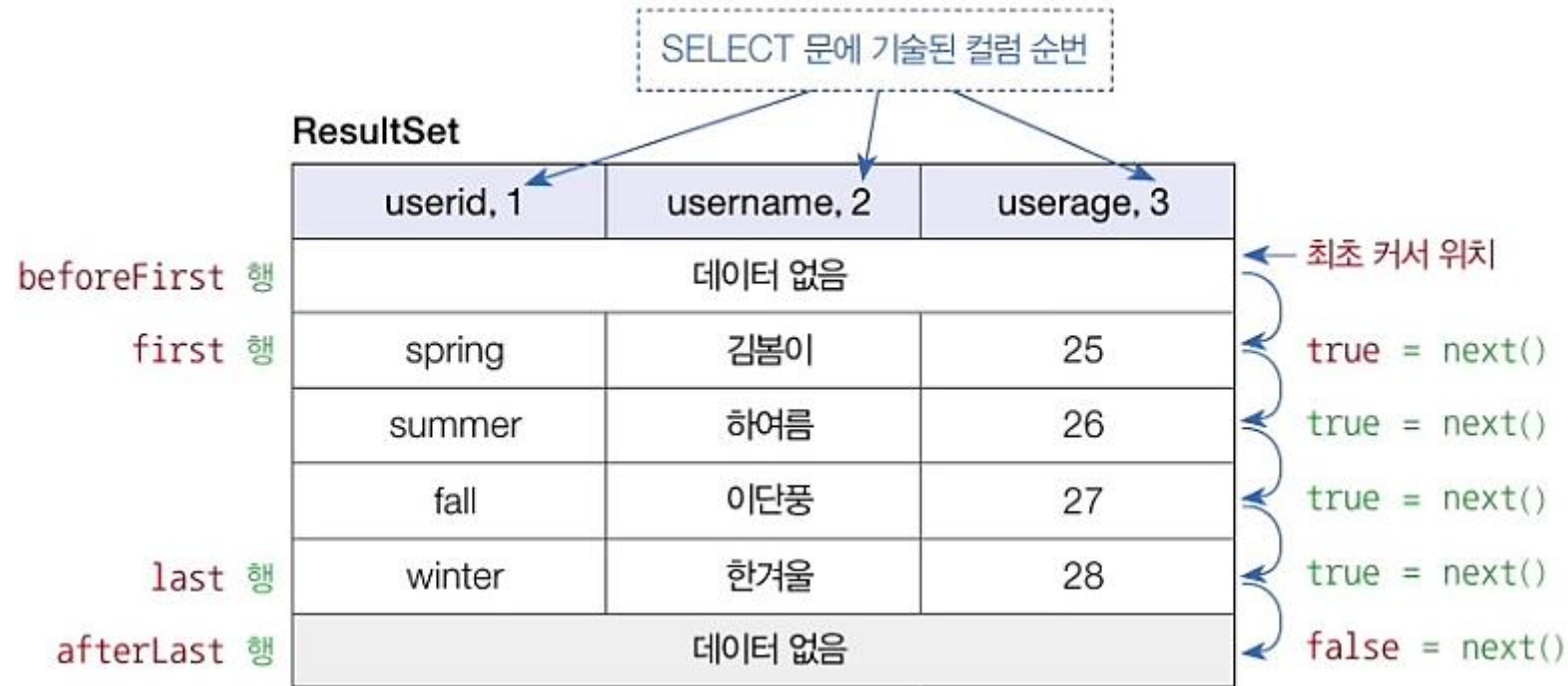
Try-catch-resources
Try(예외처리 대상 코드)
→ 자원해제, 자동close

1 JDBC 프로그래밍

✓ Statement

- SQL 문 실행 클래스
- Connection 객체를 통해 생성
Statement stmt = conn.createStatement();
- SQL 실행 메서드
 - ResultSet executeQuery(SQL문) : select문 실행
 - int executeUpdate(SQL문): insert, update, delete 문 실행

✓ ResultSet



○ 컬럼 값 추출

- getXxxx("컬러명")
 - Xxx : 추출하고자하는 데이터 타입명
 - getString(), getInt(), getLong(), getDouble()

✓ PreparedStatement

- SQL문에 값을 넣을 때 파라미터화 해서 처리

```
String sql = "INSERT INTO USERS(ID, PASSWORD, NAME, ROLE) " +  
            "VALUES(?, ?, ?, ?)";
```

- Connection 객체를 통해 생성

```
PreparedStatement pstmt = conn.prepareStatement(sql);
```

- 파라미터 설정

- pstmt.setXxx(파라미터번호, 값)
 - setString(), setInt(), setLong(), setDouble()

- SQL문 실행

```
int count = pstmt.executeUpdate();
```


✓ Statement로 Insert 문 실행하기

```
String sql = "INSERT INTO USERS(ID, PASSWORD, NAME, ROLE)" +  
            "VALUES('member2', 'member123', '일반회원', 'USER')";
```

```
int count = stmt.executeUpdate(sql);
```

○ 값을 변수로 대체한다면 ?

```
String userId = "member2";  
String password = "member123";  
String name = "일반회원";  
String role = "USER";
```

```
String sql = "INSERT INTO USERS(ID, PASSWORD, NAME, ROLE)" +  
            "VALUES('" + userId + "', '" + password + "', '" +  
            name + "', '" + role + "')";
```

→ PreparedStatement로 처리

1 JDBC 프로그래밍

 CrudTest.java

```
package org.scoula.jdbc_ex.test;

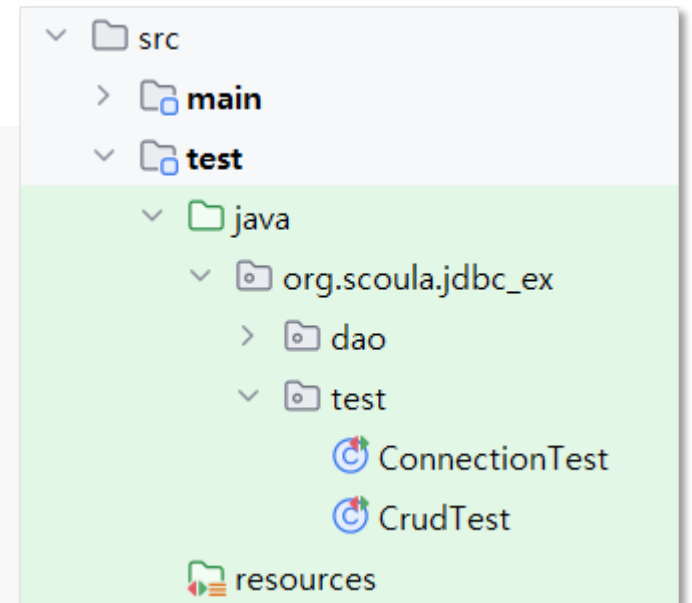
import org.junit.jupiter.api.*;
import org.scoula.jdbc_ex.common.JDBCUtil;

import java.sql.*;

@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
public class CrudTest {
    Connection conn = JDBCUtil.getConnection();

    @AfterAll
    static void tearDown() {
        JDBCUtil.close();
    }
}
```

테스트하고 싶은 기능에
순서를 주기 위한
annotation



 CrudTest.java

```
@Test
@DisplayName("새로운 user를 등록한다.")
@Order(1)
public void insertUser() throws SQLException {
    String sql = "insert into users(id, password, name, role) values(?, ?, ?, ?)";
    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setString(1, "scoula");
        pstmt.setString(2, "scoula3");
        pstmt.setString(3, "스콜라");
        pstmt.setString(4, "USER");

        int count = pstmt.executeUpdate();
        Assertions.assertEquals(1, count);
    }
}
```

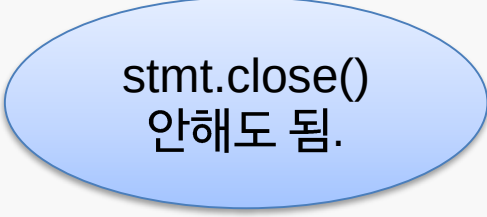
pstmt.close()
안해도 됨.

 CrudTest.java

```
@Test
@DisplayName("user 목록을 추출한다.")
@Order(2)
public void selectUser() throws SQLException {
    String sql = "select * from users";
    Statement stmt = conn.createStatement();
    ResultSet rs = stmt.executeQuery(sql);
    while(rs.next()) {
        System.out.println(rs.getString("name"));
    }
    stmt.close();
}
```

 CrudTest.java

```
@Test
@DisplayName("특정 user 검색한다.")
@Order(3)
public void selectUserById() throws SQLException {
    String userid = "scoula";
    String sql = "select * from users where id = ?";
    try(PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, userid); // where id = ?
        try(ResultSet rs = stmt.executeQuery()) {
            if(rs.next()) {
                System.out.println(rs.getString("name"));
            } else {
                throw new SQLException("scoula not found");
            }
        }
    }
}
```



stmt.close()
안해도 됨.

 CrudTest.java

```
@Test
@DisplayName("특정 user 수정한다.")
@Order(4)
public void updateUser() throws SQLException {
    String userid = "scola";
    String sql = "update users set name= ? where id = ?";
    try(PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, "스콜라 수정"); // name= ?
        stmt.setString(2, userid); // where id = ?";
        int count = stmt.executeUpdate();
        Assertions.assertEquals(1, count);
    }
}
```

pstmt.close()
안해도 됨.

 CrudTest.java

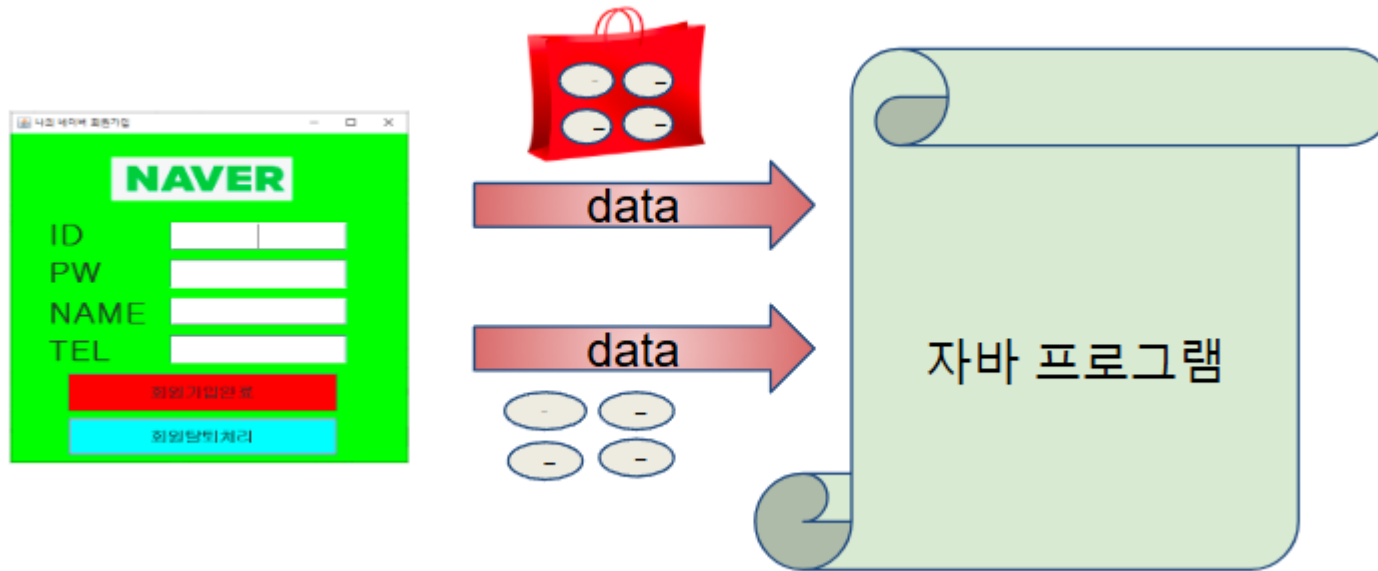
```
@Test
@DisplayName("지정한 사용자를 삭제한다.")
@Order(5)
public void deleteUser() throws SQLException {
    String userid = "scoula";
    String sql = "delete from users where id = ?";
    try(PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, userid);
        int count = stmt.executeUpdate();
        Assertions.assertEquals(1, count);
    }
}
```



pstmt.close()
안해도 됨.

1 JDBC 프로그래밍

- ✓ DTO(Data Transfer Object, VO, Value Object)
- ✓ 많은 양의 데이터를 전달할 목적으로 사용하는 가방 역할의 클래스
- ✓ 값들만 넣어서 전달하고, 전달된 값들을 꺼내어 사용하는 역할
- ✓ 자바에서는 여러 데이터를 전달할 때 하나씩 전달하지 않고, 데이터들을 넣어서 전달할 클래스(가방 역할)을 만들어 사용한다.



1 JDBC 프로그래밍

 UserVO.java

```
package org.scoula.jdbc_ex.domain;
```

```
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
```

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class UserVO {
    private String id;
    private String password;
    private String name;
    private String role;
}
```

```
*MemberVO.java X
1 package db_connect;
2
3 public class MemberVO {
4     private String id;
5     private String pw;
6     private String name;
7     private String tel;
8
9     public void setId(String id) {
10         this.id = id;
11     }
12
13     public String getId() {
14         return id;
15     }
16 }
```

데이터를 하나씩 넣을 공간인 변수 선언

변수 하나당
1) 데이터를 넣는 **set메서드**,
2) 데이터를 꺼내는 **get메서드** 정의
⇒ **Getters/Setters 메서드**라고 함.

```
@Override
public String toString() {
    return "MemberVO{" +
        "id='" + id + '\'' +
        ", pw='" + pw + '\'' +
        ", name='" + name + '\'' +
        ", tel='" + tel + '\'' +
        '}';
}
```

1 JDBC 프로그래밍

<<메서드의 입력은 여러개 나열해도 되고, vo를 사용해도 됨.>>

insert(String id, String pw, String name, String tel) - 가능

insert(MemberVO bag) - 가능

<<메서드의 반환값은 무조건 하나로 묶어주어야 함.>

return id, pw, name, tel - 불가능

return bag - 가능



앞으로, 데이터를 묶어서 전달할 경우
어떻게 해야할까??

- 같은 타입, 고정 길이 → 배열
- 다른 타입, 가변 길이 → arraylist
- db table데이터 전달 → VO

1 JDBC 프로그래밍

id: win

pw: win

name: win

tel: win

1. 드라이버 설정 ok....

2. DB연결 ok....

3. SQL문 생성 ok....

4. SQL문 전송 ok....

insert success

보내는 쪽 →

1) 가방을 만든다.

2) 가방에 넣는다.

3) 가방을 전달한다.

1

2

3

```
public class DB연결Main3 {
    public static void main(String[] args) throws Exception {
        Scanner scanner = new Scanner(System.in);
```

```
        System.out.print("id: ");
        String id = scanner.nextLine();
        System.out.print("pw: ");
        String pw = scanner.nextLine();
        System.out.print("name: ");
        String name = scanner.nextLine();
        System.out.print("tel: ");
        String tel = scanner.nextLine();
```

```
        MemberDAO dao = new MemberDAO();
```

```
        // 1. 가방을 만든다.
        MemberVO bag = new MemberVO();
```

```
        // 2. 가방에 데이터를 하나씩 넣자.
        bag.setId(id);
        bag.setPw(pw);
        bag.setName(name);
        bag.setTel(tel);
```

```
        // 3. 가방을 전달하자.
        int result = dao.insert(bag);
        if (result == 0) {
            System.out.println("insert failed");
        } else {
            System.out.println("insert success");
        }
    }
}
```

```
        scanner.close();
    }
}
```

1 JDBC 프로그래밍

받는 쪽 →

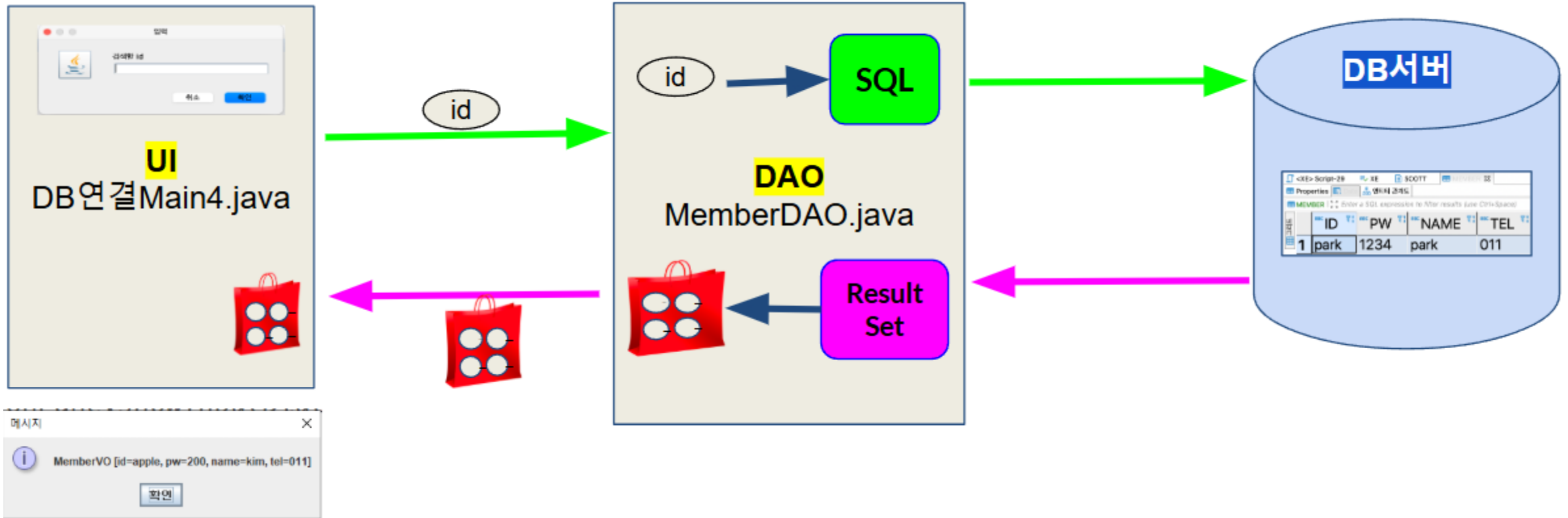
- 1) 전달받은 가방을 저장
- 2) 데이터를 하나씩 꺼낸다.

1

```
public int insert(MemberVO bag) throws Exception {  
    // 1. 드라이버(객체) 설정  
    Class.forName("com.mysql.cj.jdbc.Driver");  
    System.out.println("1. 드라이버 설정 ok....");  
  
    // 2. db연결(shop, root, 1234)  
    String url = "jdbc:mysql://localhost:3306/shop";  
    String user = "root";  
    String password = "1234";  
    Connection con = DriverManager.getConnection(url, user, password);  
    System.out.println("2. DB 연결 ok....");  
  
    // 3. sql문 생성  
    String sql = "insert into member values (?, ?, ?, ?)";  
    // String sql = "delete from member where id = '700' ";  
    // SQL문장을 담당하는 객체로 만들어주어야 자바는 인식  
    PreparedStatement ps = con.prepareStatement(sql);  
    ps.setString(1, bag.getId());  
    ps.setString(2, bag.getPw());  
    ps.setString(3, bag.getName());  
    ps.setString(4, bag.getTel());  
    System.out.println("3. SQL문 생성 ok....");  
  
    // 4. sql문 전송  
    int result = ps.executeUpdate();  
    System.out.println("4. SQL문 전송 ok....");  
  
    ps.close();  
    con.close();  
  
    return result;  
}
```

2

1 JDBC 프로그래밍



1 JDBC 프로그래밍

The screenshot shows a SQL IDE interface. At the top, there are tabs for '<none> Script-3', 'PRODUCT', 'MEMBER', and '<xe> Script-40'. The main editor displays the following SQL query:

```
SELECT * FROM "MEMBER"  
WHERE id = 'apple2'
```

Below the query, there is a 'Results 1' tab. The results are displayed in a table with the following columns: ID, PW, NAME, and TEL. The first row of data is highlighted in blue. An orange arrow points to the first row of the table, with a yellow box containing the text: 'rs.next() → 있으면, true / 없으면, false'. Another yellow box on the right side of the table contains the text: 'ResultSet rs'.

ID	PW	NAME	TEL

검색할 때는 하나의 행을/여러개의 행을 검색

자바 dto 1개 <= db테이블의 한 행

Object 관계형DB(relational DB)

0

R

M

=>ORM(object relationalDB mapping)

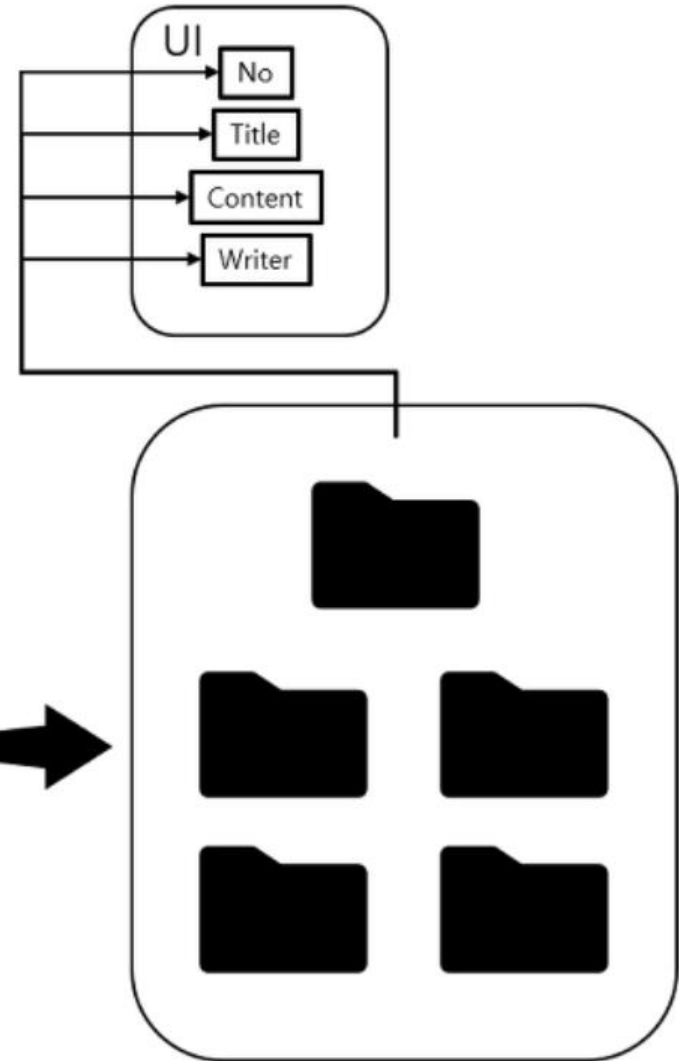
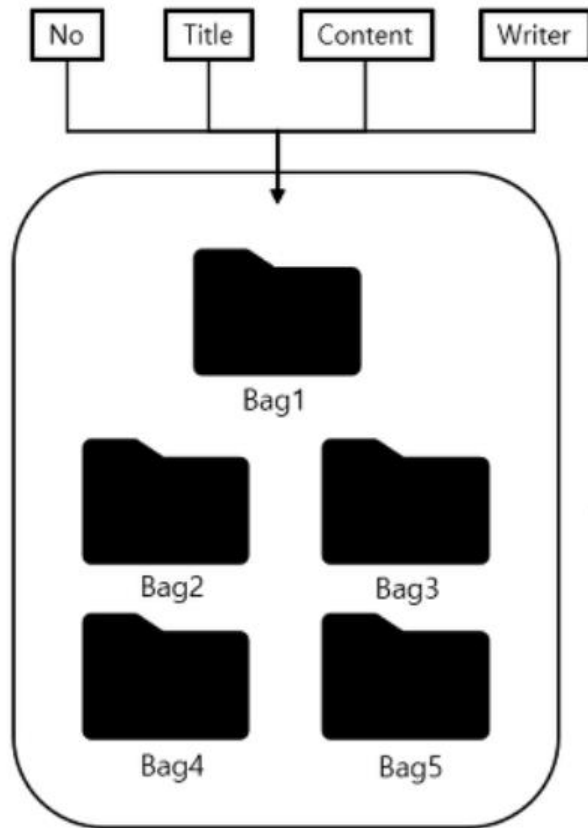
=>자바 dto클래스 하나당 RDB테이블의 한 행

=>검색 결과가 많은 경우, 가방역할의 dto들을
넣어줄 컨테이너 역할의 클래스 필요

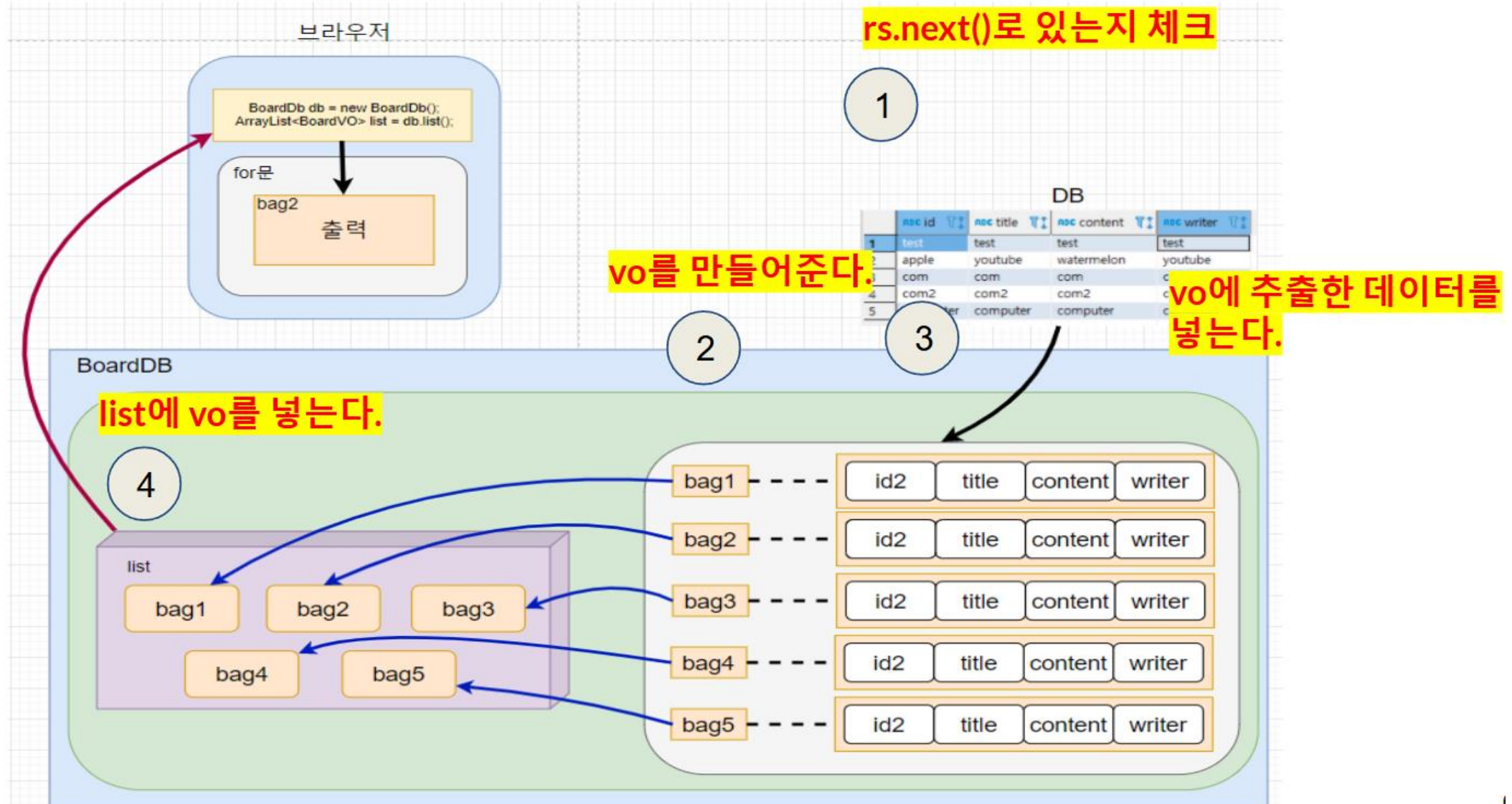
ArrayList가 컨테이너 역할

1 JDBC 프로그래밍

	123 no	ABC title	ABC content	ABC writer
1	1	cup	water	yl
2	2	benq	monitor	mb
3	3	logitech	Gpro	gy
4	4	s.Card	SoungBlaster	wl



1 JDBC 프로그래밍



1 JDBC 프로그래밍

```
import java.util.ArrayList;

public class ArrayListTest1 {

    public static void main(String[] args) {
        ArrayList list = new ArrayList();
        list.add("홍길동");
        list.add(100);
        list.add(11.11);
        System.out.println(list.get(0));
        System.out.println(list.get(1));
        System.out.println(list.get(2));
        System.out.println(list.size());
        list.remove(0);
        System.out.println(list.size());

        for (int i = 0; i < list.size(); i++) {
            System.out.println(i + ": " + list.get(i));
        }
    }
}
```

ArrayList

- 넣을 때는 add()
- 꺼낼 때는
get(인덱스)

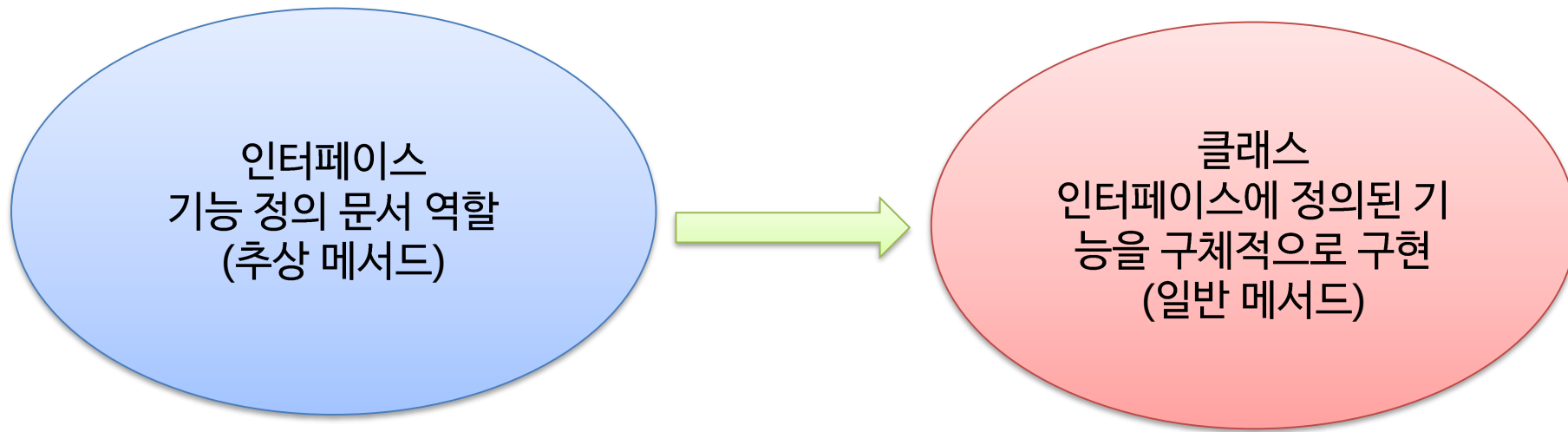
1 JDBC 프로그래밍

✓ DAO 패턴 적용

○ DAO 클래스

- Data Access Object
- 데이터베이스에 접근하여 실질적인 데이터베이스 연동 작업을 담당하는 클래스
- 테이블에 대한 CRUD 연산을 처리

○ 인터페이스 정의 후 구현 클래스 작성



 UserDao.java

```
package org.scoula.jdbc_ex.dao;

import org.scoula.jdbc_ex.domain.UserVO;

import java.sql.SQLException;
import java.util.List;
import java.util.Optional;

public interface UserDao {
    // 회원 등록
    int create(UserVO user) throws SQLException;

    // 회원 목록 조회
    List<UserVO> getList() throws SQLException;

    // 회원 정보 조회
    UserVO get(String id) throws SQLException;

    // 회원 수정
    int update(UserVO user) throws SQLException;

    // 회원 삭제
    int delete(String id) throws SQLException;
}
```

 UserDaoImpl.java

```
package org.scoula.jdbc_ex.dao;

import org.scoula.jdbc_ex.common.JDBCUtil;
import org.scoula.jdbc_ex.domain.UserVO;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class UserDaoImpl implements UserDao {
    Connection conn = JDBCUtil.getConnection();

    // USERS 테이블 관련 SQL 명령어
    private String USER_LIST = "select * from users";
    private String USER_GET = "select * from users where id = ?";
    private String USER_INSERT = "insert into users values(?, ?, ?, ?)";
    private String USER_UPDATE = "update users set name = ?, role = ? where id = ?";
    private String USER_DELETE = "delete from users where id = ?";
```

 UserDaoImpl.java

```
// 회원 등록
@Override
public int create(UserVO user) throws SQLException {
    PreparedStatement stmt = conn.prepareStatement(USER_INSERT);
    stmt.setString(1, user.getId());
    stmt.setString(2, user.getPassword());
    stmt.setString(3, user.getName());
    stmt.setString(4, user.getRole());
    int count = stmt.executeUpdate();
    stmt.close();
    return count;
} //create
```

```
private String USER_INSERT = "insert into users values(?, ?, ?, ?)";
```

 UserDaoImpl.java

```
// 회원 목록 조회
```

```
@Override
```

```
public List<UserVO> getList() throws SQLException {
```

```
    List<UserVO> userList = new ArrayList<>();
```

```
    PreparedStatement stmt = conn.prepareStatement(USER_LIST);
```

```
    ResultSet rs = stmt.executeQuery();
```

```
    while (rs.next()) {
```

```
        UserVO user = new UserVO();
```

```
        user.setId(rs.getString("ID"));
```

```
        user.setPassword(rs.getString("PASSWORD"));
```

```
        user.setName(rs.getString("NAME"));
```

```
        user.setRole(rs.getString("ROLE"));
```

```
        userList.add(user);
```

```
    } //while
```

```
    rs.close(); stmt.close();
```

```
    return userList;
```

```
} //getList
```

```
private String USER_LIST = "select * from users";
```

 UserDaoImpl.java

```
// 회원 정보 조회
@Override
public UserVO get(String id) throws SQLException {
    UserVO user = new UserVO();
    PreparedStatement stmt = conn.prepareStatement(USER_GET);
    stmt.setString(1, id);

    ResultSet rs = stmt.executeQuery();
    if (rs.next()) {
        user.setId(rs.getString("ID"));
        user.setPassword(rs.getString("PASSWORD"));
        user.setName(rs.getString("NAME"));
        user.setRole(rs.getString("ROLE"));
    } //if
    rs.close(); stmt.close();
    return user;
} //get
```

```
private String USER_GET = "select * from users where id = ?";
```


 UserDaoImpl.java

```
// 회원 수정
@Override
public int update(UserVO user) throws SQLException {
    PreparedStatement stmt = conn.prepareStatement(USER_UPDATE);
    stmt.setString(1, user.getName());
    stmt.setString(2, user.getRole());
    stmt.setString(3, user.getId());
    int count = stmt.executeUpdate();
    stmt.close();
    return count;
} //update

// USERS 테이블 관련 CRUD 메소드
// 회원 삭제
@Override
public int delete(String id) throws SQLException {
    PreparedStatement stmt = conn.prepareStatement(USER_DELETE);
    stmt.setString(1, id);
    int rows = stmt.executeUpdate();
    stmt.close();
    return rows;
} //delete
} //class
```

```
private String USER_UPDATE = "update users set name = ?, role = ? where id = ?";
```

```
private String USER_DELETE = "delete from users where id = ?";
```

1 JDBC 프로그래밍

✓ UserDao 테스트 클래스

```
public interface UserDao {  
    // 회원 등록  
    int create(UserVO user)  
  
    // 회원 목록 조회  
    List<UserVO> getList()  
  
    // 회원 정보 조회  
    Optional<UserVO> get(S  
  
    // 회원 수정  
    int update(UserVO user)  
  
    // USERS 테이블 관련 CRUD  
    // 회원 삭제  
    int delete(String id)
```

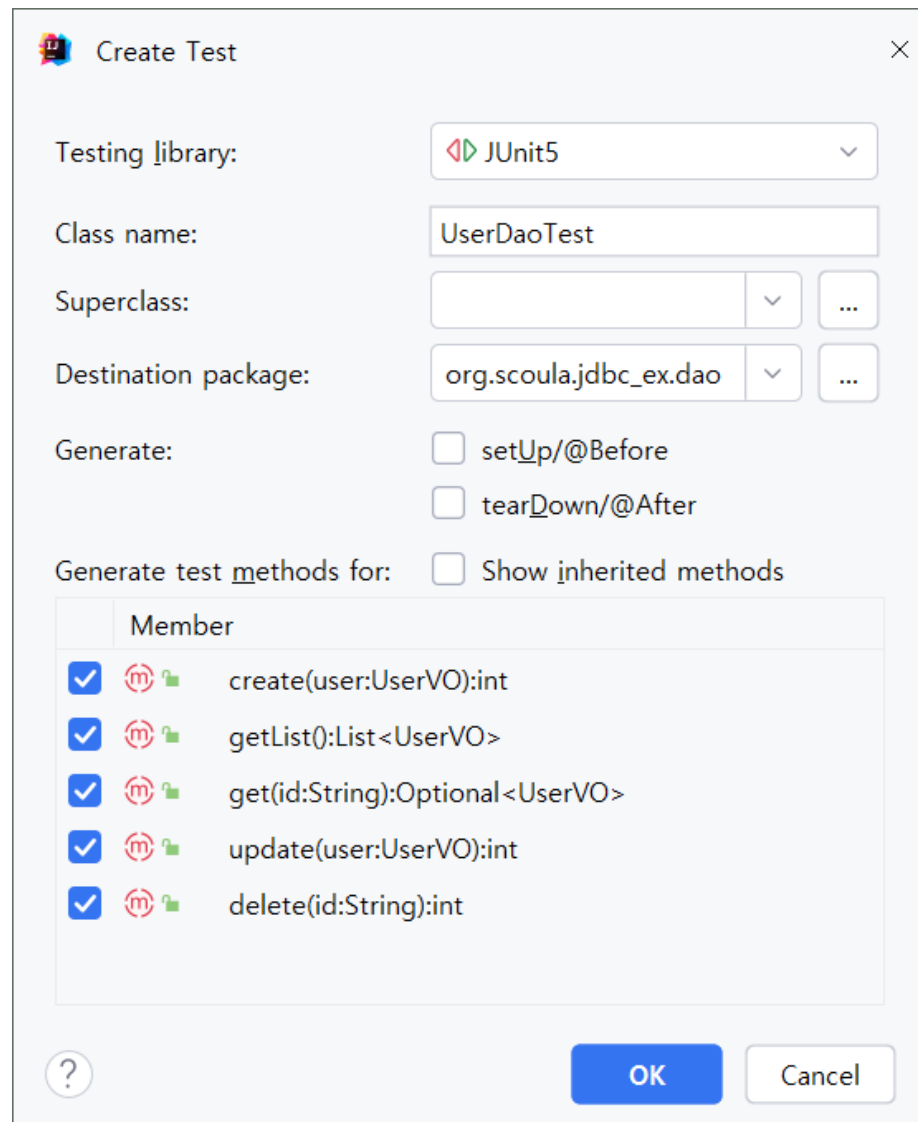
Action	Shortcut
Show Context Actions	Alt+Enter
Paste	Ctrl+V
Copy / Paste Special	>
Column Selection Mode	Alt+Shift+Insert
Find Usages	Alt+F7
Go To	>
Folding	>
Analyze	>
Refactor	>
Generate...	Alt+Insert
Open In	>
Navigation Bar	Alt+Home
Declaration or Usages	Ctrl+B
Implementation(s)	Ctrl+Alt+B
Type Declaration	Ctrl+Shift+B
Super Class or Interface	Ctrl+U
Related Symbol...	Ctrl+Alt+Home
Test	Ctrl+Shift+T

Choose Test for UserDao (0 found)

Create New Test...

1 JDBC 프로그래밍

✓ UserDao 테스트 클래스



The image shows a 'Create Test' dialog box with the following fields and options:

- Testing library: JUnit5
- Class name: UserDaoTest
- Superclass: (empty)
- Destination package: org.scoula.jdbc_ex.dao
- Generate: ☐ setUp/@Before, ☐ tearDown/@After
- Generate test methods for: ☐ Show inherited methods

Member		
☒		create(user:UserVO):int
☒		getList():List<UserVO>
☒		get(id:String):Optional<UserVO>
☒		update(user:UserVO):int
☒		delete(id:String):int

Buttons: ? (help), OK, Cancel

UserDaoTest.java

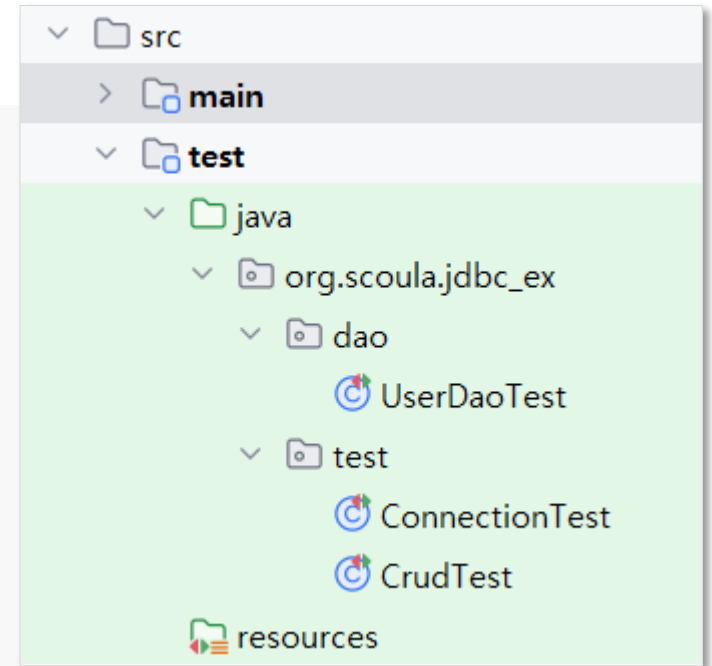
```
package org.scoula.jdbc_ex.dao;

import org.junit.jupiter.api.*;
import org.scoula.jdbc_ex.common.JDBCUtil;
import org.scoula.jdbc_ex.domain.UserVO;

import java.sql.SQLException;
import java.util.List;
import java.util.NoSuchElementException;

@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
class UserDaoTest {
    UserDao dao = new UserDaoImpl();

    @AfterAll
    static void tearDown() {
        JDBCUtil.close();
    }
}
```



 UserDaoTest.java

```
@Test
@DisplayName("user를 등록합니다.")
@Order(1)
void create() throws SQLException {
    UserVO user = new UserVO("ssamz3", "ssamz123", "쌤즈", "ADMIN");
    int count = dao.create(user);
    Assertions.assertEquals(1, count);
}

@Test
@DisplayName("UserDao User 목록을 추출합니다.")
@Order(2)
void getList() throws SQLException {
    List<UserVO> list = dao.getList();
    for(UserVO vo: list) {
        System.out.println(vo);
    }
}
```

 UserDaoTest.java

```
@Test
@DisplayName("특정 user 1건을 추출합니다.")
@Order(3)
void get() throws SQLException {
    UserVO user = dao.get("ssamz3");
    Assertions.assertNotNull(user);
}

@Test
@DisplayName("user의 정보를 수정합니다.")
@Order(4)
void update() throws SQLException {
    UserVO user = dao.get("ssamz4");
    user.setName("쌤즈33");
    int count = dao.update(user);
    Assertions.assertEquals(1, count);
}

@Test
@DisplayName("user를 삭제합니다.")
@Order(5)
void delete() throws SQLException {
    int count = dao.delete("ssamz4");
    Assertions.assertEquals(1, count);
}
```

1 JDBC 프로그래밍

 UserMain.java

```
package org.scoula.jdbc_ex;

import org.scoula.jdbc_ex.dao.UserDao;
import org.scoula.jdbc_ex.dao.UserDaoImpl;
import org.scoula.jdbc_ex.domain.UserVO;
import java.sql.SQLException;
import java.util.List;
import java.util.Scanner;

public class UserMain {
    static UserDao userDao = new UserDaoImpl();
    static Scanner sc = new Scanner(System.in);

    public static void main(String[] args) throws SQLException {
        while (true) {
            System.out.println("원하는 작업을 선택하십시오.");
            System.out.println("=====");
            System.out.println("1. insert");
            System.out.println("2. selectList");
            System.out.println("3. selectOne");
            System.out.println("4. update");
            System.out.println("5. delete");
            System.out.println("6. exit");
            System.out.println("=====");
```

```
        System.out.print("번호 입력)>> ");
        int choice = sc.nextInt();
        if (choice == 1) {
            insert();
        } else if (choice == 2) {
            selectList();
        } else if (choice == 3) {
            selectOne();
        } else if (choice == 4) {
            update();
        } else if (choice == 5) {
            delete();
        } else if (choice == 6) {
            System.out.println("프로그램을 종료합니다.");
            sc.close();
            JDBCUtil.close();
            System.exit(0); //프로그램 종료
        } else {
            System.out.println("선택이 올바르지 않음.");
        }
    }

    private static void printVO(UserVO userVO) {
        System.out.println("아이디 : " + userVO.getId() + ", 패스워드 : " + userVO.getPassword()
            + ", 이름 : " + userVO.getName() + ", 역할 : " + userVO.getRole());
    }
}
```

 UserMain.java

```

public static void insert() throws SQLException {
    UserVO user = new UserVO();

    System.out.println("id, pw, name, role을 순서대로 입력하시오.");
    System.out.print("id(12글자 이내) >> ");
    user.setId(sc.next());
    System.out.print("pw(12글자 이내) >> ");
    user.setPassword(sc.next());
    System.out.print("name(30글자 이내) >> ");
    user.setName(sc.next());
    System.out.print("role(6글자 이내) >> ");
    user.setRole(sc.next());

    int result = userDao.create(user);

    if (result == 1) {
        System.out.println("insert 성공!");
    }
}

```

```

public static void selectList() throws SQLException {
    List<UserVO> list = userDao.getList();

    if (list.size() == 0) {
        System.out.println("검색 실패, 데이터없음.");
    } else {
        for (UserVO userVO : list) {
            printVO(userVO);
        }
    }
}

public static void selectOne() throws SQLException {
    System.out.println("검색하고자하는 id를 입력하시오.");
    System.out.print("id(12글자 이내) >> ");
    String id = sc.next();

    UserVO userVO = userDao.get(id);

    if (userVO.getId() == null) {
        System.out.println("검색 실패, 없는 id입니다.!");
    } else {
        printVO(userVO);
    }
}

```


 UserMain.java

```
public static void update() throws SQLException {
```

```
    //가방 만들고
```

```
    UserVO user = new UserVO();
```

```
    //입력받아 가방에 넣고
```

```
    System.out.println("name, role, id를 순서대로 입력하시오.");
```

```
    System.out.print("name(30글자 이내) >> ");
```

```
    user.setName(sc.next());
```

```
    System.out.print("role(6글자 이내) >> ");
```

```
    user.setRole(sc.next());
```

```
    System.out.print("id(12글자 이내) >> ");
```

```
    user.setId(sc.next());
```

```
    //dao의 update메서드 호출하면서 가방 전달하고
```

```
    int result = userDao.update(user);
```

```
    //결과 처리
```

```
    if (result == 1) {
```

```
        System.out.println("update 성공!");
```

```
    }else {
```

```
        System.out.println("update 실패!");
```

```
    }
```

```
}
```

```
public static void delete() throws SQLException {
```

```
    System.out.println("삭제하고자하는 id를 입력하시오.");
```

```
    System.out.print("id(12글자 이내) >> ");
```

```
    String id = sc.next();
```

```
    int result = userDao.delete(id);
```

```
    //결과 처리
```

```
    if (result == 1) {
```

```
        System.out.println("delete 성공!");
```

```
    }else {
```

```
        System.out.println("delete 실패!");
```

```
    }
```

```
}
```

```
}
```